

Software Re-Engineering

Comprehensive Study Guide: Past Papers & Book Questions

Part 1: Past Papers

MID 1 SPRING 2026 • QUESTION # 1

Q: Differentiate between Reverse Engineering and Forward Engineering in the context of software re-engineering. How do they complement each other?

- **Forward Engineering:** The traditional process of moving from high-level abstractions and logical, implementation-independent designs down to the physical implementation of a system.
- **Reverse Engineering:** The process of analyzing a subject system to identify its components and their interrelationships, and creating representations of the system in another form or at a higher level of abstraction without changing the underlying software.
- **How they complement each other:** In the reengineering lifecycle, reverse engineering and forward engineering form two complementary halves connected by a transformation phase (expressed as $\text{Reengineering} = \text{Reverse Engineering} + \Delta + \text{Forward Engineering}$). Reverse engineering is a prerequisite because one cannot transform what is not understood; it recovers designs, architectures, and requirements from legacy code. Once modifications (Δ) are applied to these abstract representations, forward engineering takes over to refine and generate the concrete physical implementation of the target system.

Q: Differentiate between software maintenance and software re-engineering in terms of objectives and activities involved.

• **Objectives:**

- *Software Maintenance:* Primarily aims to keep an operational system running by fixing bugs, making minor enhancements, and preserving existing functionality without significantly altering the system's core architecture.
- *Software Re-Engineering:* A more radical, major effort aimed at significantly reducing the complexity of a legacy system so that it can continue to be adapted at an acceptable cost, raising quality levels, and preparing the system for major future extensions or platform migrations.

• **Activities Involved:**

- *Software Maintenance:* Event-driven routine activities consisting of bug fixes, corrective maintenance, minor requirement enhancements, updating documentation, and localized regression testing.
- *Software Re-Engineering:* Multi-phased lifecycle activities encompassing full reverse engineering (program comprehension, architecture recovery), structural alteration (refactoring code, architectural remodularization, data re-engineering), and forward engineering to reconstitute the system into a new form.

Q: Describe the concept of Technical Debt in the context of Software Re-Engineering. Give an example of how it may occur?

- **Concept:** Technical debt represents the implied future cost of additional rework, maintenance, and prolonged development caused by choosing an easy, quick-fix, or expedient solution now instead of a well-engineered design. Over time, recurring ad-hoc modifications degrade software structure (design drift and increasing complexity), making subsequent changes progressively more expensive until reengineering becomes necessary to pay off the debt.
- **Example of Occurrence:** Developers under strict release deadlines copy-and-paste existing code blocks across multiple modules (code duplication) rather than designing an abstracted, reusable method or hierarchy. Another typical example is implementing nested conditional switch/case statements to handle new types instead of refactoring to polymorphism, which creates brittle, hard-to-maintain code.

Q: Reverse Engineering is a key activity in the re-engineering exercises. Explain the difference between top-down and bottom-up techniques for reverse engineering purpose.

- **Top-Down Technique:** Starts from a high-level representation, conceptual hypothesis, or domain expectation and verifies it against the source code (e.g., the *Speculate about Design* pattern or the software reflection model). The engineer formulates an architectural concept and inspects the code to confirm or refute it.
- **Bottom-Up Technique:** Starts directly from the concrete source code or database schema, filters out relevant entities, and aggregates/chunks them into higher-level abstractions and structural models (e.g., *Analyze the Persistent Data* and *Study the Exceptional Entities*).

Q: Restructuring the architecture is an important activity in a Software Re-Engineering effort. How can Model-Driven Development approaches facilitate this aspect? Also describe possible challenges.

- **How MDD Facilitates Architectural Restructuring:** Model-Driven Development (MDD) elevates the level of abstraction from code to models. In reengineering, reverse engineering extracts platform-independent models (PIM) from legacy code. Architectural transformations (such as modularization, layering, or migrating to microservices) can then be modeled formally using model-transformation rules, after which automated code-generation tools produce the new platform-specific target implementation (PSM), significantly reducing manual recoding errors.
- **Possible Challenges:**
 1. *Imperfect Model Recovery:* Automatically extracting complete, high-fidelity models that capture legacy business rules and implicit dependencies from source code is difficult.
 2. *Semantic Gap and Information Loss:* Crucial non-functional attributes, optimizations, or platform idioms can be lost during abstraction into high-level models.
 3. *Complexity of Transformation Rules:* Specifying complete and robust transformation rules for complex, heterogeneous legacy systems requires specialized tools and deep expertise.

Part 2: Book Questions

SOFTWARE EVOLUTION AND MAINTENANCE, CHAPTER 1 • QUESTION # 1

Q: Discuss the differences between software evolution and software maintenance.

- **Software Maintenance:** Primarily means preventing software from failing to deliver intended functionality through bug fixing and support activities carried out *after* delivery. It is planned and event-driven, focusing on preserving functionality and keeping an installed system running with little to no change to its core architecture or design.
- **Software Evolution:** Means a continual change from a lesser, simpler, or worse state to a higher or better state. It encompasses all activities carried out to effect changes in requirements, reflecting the sustained growth and adaptation of the system over time.

SOFTWARE EVOLUTION AND MAINTENANCE, CHAPTER 1 • QUESTION # 2

Q: Explain why a software system which is used in a real-world environment must be changed to not become progressively less useful.

According to Lehman's First Law of Software Evolution (*Continuing Change*), a software system operating in a real-world environment (an E-type system) contains embedded assumptions about that environment. As users utilize the software, they gain experience, their expectations grow, and the operational business environment changes. Consequently, many original assumptions become invalid. Unless the system is continually modified and adapted to satisfy these emerging needs, it progressively diverges from real-world requirements and becomes less useful.

SOFTWARE EVOLUTION AND MAINTENANCE, CHAPTER 1 • QUESTION # 3

Q: What are some characteristics of maintaining software as opposed to new software systems?

1. *Constraints of an Existing System:* Modifications must strictly adhere to and remain compatible with the preexisting architecture, design, and source code.
2. *Event-Driven Nature:* Maintenance is triggered by unscheduled events (bug reports, user change requests), whereas new development is requirements-driven.
3. *Shorter Time Frame:* Maintenance tasks typically span from a few hours to a few months, unlike new development projects which span one or more years.
4. *Availability of Preexisting Test Data:* Maintainers can reuse and select a subset of preexisting test suites for regression testing rather than writing everything from scratch.

Q: You are asked to make a change to a system that leaves its functional specification unchanged but affects the design and source code of the system. This can be any of the four types of maintenance mentioned earlier except one. Identify the exception and justify your answer.

- (a) Corrective maintenance
- (b) Adaptive maintenance
- (c) Perfective maintenance
- (d) Preventive maintenance

- **Exception:** (b) Adaptive maintenance.
- **Justification:** Adaptive maintenance modifies software to interface with a changing or changed external environment (e.g., new operating system, hardware platform, or external communication protocols), which changes the operational specifications. In contrast, corrective maintenance fixes defects to satisfy existing functional specifications, perfective maintenance improves performance or internal maintainability without altering functionality, and preventive maintenance restructures code to avert future failures while keeping external functional specifications intact.

Q: Discuss the major differences between COTS-based software development and traditional in-house software development activities.

- **Source Code Visibility & Modification:** In-house development allows full access to internal source code and design documentation for debugging and direct modification. COTS components are commercial black boxes delivered without source code, preventing direct modifications.
- **Integration vs. Construction:** In-house systems are built from modular code, whereas COTS development centers on component selection, writing wrappers, glue code, and tailoring scripts.
- **Vendor Dependency:** COTS evolution depends entirely on third-party vendor release and patch schedules, complicating maintenance planning across multiple components.

Q: One of the key sources of risks in COTS-based development is the reliance on one or more third-party software vendors. However, this dependence can also present new challenges for the evolution of such systems. Which of the following evolution challenges can be directly attributed to reliance on the vendor?

- (a) Lack of control over when errors in components are fixed.
- (b) Number and complexity of inter-component interfaces.
- (c) Diversity of inter-component interfaces.
- (d) Lack of experience and tools for evolving COTS-based systems.

- **Answer:** (a) Lack of control over when errors in components are fixed.
- **Justification:** Because COTS components are proprietary black boxes maintained by external suppliers, maintainers cannot directly inspect or modify the code to resolve bugs and must rely on the vendor's maintenance schedule.

Q: What are the objectives of SCM?

According to IEEE 1042 and textbook Chapter 1, the core objectives of Software Configuration Management (SCM) are:

1. Uniquely identify every version of every software configuration item at various points in time.
2. Retain past versions of software code and technical documentation.
3. Provide an audit trail for all modifications performed throughout the lifecycle.
4. Maintain the traceability and integrity of system changes, ensuring released software is not contaminated by uncontrolled or unapproved changes.

Q: A feature of any complex change to an existing software system is that it is likely to introduce new defects, even if the aim of the change is to remove defects. When considering whether or not to implement a change request, should this feature be considered as a cost, benefit, or risk associated with the change request?

- **Answer:** It must be considered as a **risk**.
- **Justification:** The introduction of unintended ripple-effect defects is an uncertain, potential negative consequence that threatens system reliability, requiring rigorous risk estimation and change impact analysis prior to approving a change request.

Q: System A is a mission critical legacy system that captures and stores detailed data on product sales. Data from system A must be regularly extracted and loaded into a new system (B), which is to be used to help managers understand the changes in sales patterns from week to week. Initial estimates suggest that the data for 1 week can be extracted and transformed in around 3 hours. What migration frequency would you choose for this new application?

- (a) Migrate on update.
- (b) Migrate daily, every evening at 2.00 a.m.
- (c) Migrate weekly, every Sunday evening at 2.00 a.m.
- (d) Migrate monthly, on the last Sunday of every month at 2.00 a.m.

- **Answer:** (c) Migrate weekly, every Sunday evening at 2.00 a.m.
- **Justification:** The primary purpose of System B is to analyze sales pattern variations specifically on a *week-to-week* basis. Since processing a single week's data requires only 3 hours, executing the migration batch weekly during off-peak hours (Sunday evening at 2:00 a.m.) meets user requirements while avoiding unnecessary daily operational load on mission-critical System A.

Q: What are some of the risks of not doing an impact analysis before effecting a change?

1. *Unforeseen Ripple Effects*: Unanalyzed modifications can propagate faults into dependent modules, breaking previously functional features.
2. *Underestimating Cost and Schedule*: Maintenance teams cannot accurately forecast required staffing, budget, or timeframe, leading to project delays.
3. *System Inconsistency and Architectural Degradation*: Uncontrolled changes can violate interface assumptions and corrupt system baselines.
4. *Inadequate Regression Testing*: Without knowing which components are impacted, testing teams fail to design focused test suites.

Q: What actions can be taken to minimize the impact of fixing defects?

1. *Perform Impact Analysis*: Determine both the Starting Impact Set (SIS) and potential ripple effects before altering code.
2. *Enforce High Cohesion & Information Hiding*: Encapsulate data and functions so that changes are strictly localized.
3. *Continuous Refactoring*: Restructure code to eliminate smells and simplify logic prior to injecting fixes.
4. *Automated Regression Testing*: Run unit and system regression test suites immediately following modifications to catch regressions.
5. *Strict Configuration Management*: Ensure all defect changes are verified, reviewed, and tracked via change control procedures.

Q: What problems do maintainers face when rewriting or reengineering a piece of code? What are the causes of those problems?

• **Problems Faced:**

1. High risk of introducing new defects into previously working features.
2. Great difficulty in comprehending original operational behavior and business logic.
3. Regression failures during integration with non-reengineered legacy modules.

• **Causes:**

1. Out-of-date, inconsistent, or missing technical documentation and specifications.
2. Design drift and architectural degradation accumulated from years of unstructured maintenance changes.
3. Departure of the original developers and loss of domain knowledge.
4. High coupling and lack of automated baseline test cases.

Q: Explain the term hypotheses in the context of program understanding.

In program comprehension, a hypothesis is a tentative conjecture formulated by a programmer regarding the computational goals, domain concepts, or structural behaviors implemented by a piece of code. Programmers construct hypotheses by reading code text, identifiers, comments, and beacons, and subsequently verify, refine, or refute these conjectures through top-down or bottom-up reading to construct an accurate mental model.

Q: What benefits can be derived from reusing software?

1. *Increased Productivity*: Reusing existing components frees software engineers from developing routine functionalities from scratch.
2. *Reduced Time-to-Market*: Pre-existing assets significantly shorten the overall development and deployment cycle.
3. *Higher Quality and Reliability*: Reused artifacts from previous releases or libraries have already undergone extensive operational testing and defect removal.
4. *Reduced Maintenance Costs*: Leveraging proven components lowers overall fault density and decreases long-term lifecycle expenditures.

Q: Why do we Reengineer?

Legacy software is critical and valuable to business operations, but it cannot be easily replaced or upgraded except at high cost. The goal of reengineering is to reduce system complexity sufficiently so that it can continue to be used and adapted at an acceptable cost. Specific drivers include:

1. *Unbundling Monolithic Systems*: Allowing components to be marketed separately or recombined flexibly.
2. *Improving Performance*: Restructuring and cleaning up architecture before performance tuning ("first do it, then do it right, then do it fast").
3. *Porting to New Platforms*: Separating platform-dependent code from core logic.
4. *Design Extraction*: Recovering the underlying design as a basis for reimplementation.
5. *Exploiting New Technology*: Adopting modern standards, frameworks, or libraries to reduce maintenance costs.
6. *Reducing Human Dependencies*: Documenting implicit system knowledge to make future evolution less dependent on specific experts.

Q: Which parts of a legacy system should you reengineer?

According to the core reengineering lifecycle patterns (*If It Ain't Broke, Don't Fix It* and *Most Valuable First*), organizations cannot afford to rebuild everything. Reengineering efforts must focus only on components that are "broken" from a maintenance perspective—meaning parts that are putting the legacy at risk because they must be frequently adapted to changing requirements but have become difficult to modify due to design drift and high complexity, or components that contain high defect concentrations. Components that are stable and function reliably without incurring excessive maintenance costs should be wrapped or left alone, regardless of their internal code quality.